

CORRIGÉ DE L'ÉPREUVE

Exercice 1 : Définitions

- **Processeur logique :**

Un processeur logique (ou CPU logique) est une unité de traitement virtuelle vue par le système d'exploitation. Dans un système multicoeur avec hyperthreading, un seul cœur physique peut présenter deux processeurs logiques au SE, permettant l'exécution simultanée de deux flots d'instructions.

- **Processus :**

Un processus est un programme en cours d'exécution. Il constitue l'unité d'allocation de ressources du SE : il possède son propre espace d'adressage, ses fichiers ouverts, son contexte d'exécution (registres, compteur de programme) et est décrit par un PCB (Process Control Block).

- **Thread :**

Un thread (ou fil d'exécution) est la plus petite unité d'exécution au sein d'un processus. Tous les threads d'un même processus partagent le même espace d'adressage et les ressources du processus, mais possèdent chacun leur propre pile et registres. Les threads permettent la concurrence interne à un processus.

- **Instruction :**

Une instruction est une opération élémentaire qu'un processeur peut exécuter directement. Elle est encodée en langage machine (suite de bits) et réalise une action atomique : opération arithmétique, transfert mémoire, saut conditionnel, etc.

- **Programme :**

Un programme est un ensemble d'instructions stockées sur disque (fichier exécutable), décrivant de façon statique un traitement à effectuer. Un programme devient un processus lorsqu'il est chargé en mémoire et mis en exécution par le système d'exploitation.

- **Simultanéité :**

La simultanéité (ou concurrence) désigne la capacité d'un système à gérer plusieurs activités en même temps. Elle peut être réelle (parallélisme sur plusieurs cœurs) ou apparente (entrelacement rapide des exécutions sur un seul cœur, donnant l'illusion de l'exécution simultanée).

Exercice 2 : Processus

2.1) Rôle du PCB (Process Control Block)

Le PCB est la structure de données centrale que le SE maintient pour chaque processus. Il joue le rôle de *carte d'identité* du processus et contient notamment :

- L'identifiant du processus (PID) et du processus parent (PPID) ;
- L'état courant du processus (prêt, en cours, bloqué, etc.) ;
- Le contexte d'exécution : valeurs des registres, compteur ordinal (PC), pointeur de pile (SP) ;
- Les informations de gestion mémoire : table des pages, segments, limites ;
- Les ressources allouées : fichiers ouverts, sémaphores, E/S en cours ;
- Les informations d'ordonnancement : priorité, temps CPU consommé.

Le PCB permet au SE de sauvegarder et restaurer le contexte d'un processus lors d'une commutation de contexte.

2.2) États d'un processus

Un processus peut passer par cinq états :

- **Nouveau (New)** : Le processus vient d'être créé, son PCB est initialisé.
- **Prêt (Ready)** : Le processus est chargé en mémoire et attend qu'on lui alloue le CPU.
- **En exécution (Running)** : Le processus dispose du CPU et exécute ses instructions.
- **Bloqué/En attente (Waiting)** : Le processus attend la fin d'une E/S ou d'un événement externe.
- **Terminé (Terminated)** : Le processus a fini son exécution ; ses ressources sont libérées.

2.3) Rôle de l'ordonnanceur (Scheduler)

L'ordonnanceur est le composant du noyau chargé de décider quel processus prêt doit obtenir le CPU et pendant combien de temps. Ses rôles principaux sont :

- Sélectionner le prochain processus à exécuter parmi la file des processus prêts, selon un algorithme (FCFS, SJF, Round Robin, priorités...);
- Réaliser la commutation de contexte : sauvegarder le PCB du processus sortant et restaurer celui du processus entrant ;
- Optimiser l'utilisation du CPU, le temps de réponse et l'équité entre processus.

Exercice 3 : Synchronisation des processus (voie ferrée)

La voie ferrée est une ressource critique partagée : un seul sens de circulation est autorisé à la fois. On utilise un sémaphore **voie** initialisé à 1 (exclusion mutuelle) pour protéger l'accès.

Solution avec sémaphore :

```
/* Initialisation */
sémaphore voie = 1 ; /* 1 = voie libre, 0 = voie occupée */

/* ■■■■ Processus Train AversB ■■■■ */
Train_AversB() {
P(voie) ; /* Demande d'accès à la voie par A */
/* Section critique : circulation de A vers B */
Circulation_A_vers_B() ;
V(voie) ; /* Sortie de la voie par B */
}

/* ■■■■ Processus Train BversA ■■■■ */
Train_BversA() {
P(voie) ; /* Demande d'accès à la voie par B */
/* Section critique : circulation de B vers A */
Circulation_B_vers_A() ;
V(voie) ; /* Sortie de la voie par A */
}
```

Explication :

- **P(voie)** : opération d'attente — si voie = 0 (voie occupée), le train se bloque ; sinon voie devient 0.
- **V(voie)** : opération de libération — voie repasse à 1, débloquent un éventuel train en attente.

- L'exclusion mutuelle garantit qu'un seul train circule à la fois, dans un sens ou dans l'autre, respectant la contrainte de la voie unique.

Exercice 4 : Noyau système

4.1) Les trois parties composant le noyau du système

- 1. Le gestionnaire de processus** : Crée, détruit et ordonnance les processus ; gère la synchronisation et la communication inter-processus (IPC).
- 2. Le gestionnaire de mémoire** : Alloue et libère la mémoire physique et virtuelle ; gère la pagination, la segmentation et la mémoire virtuelle.
- 3. Le gestionnaire d'entrées/sorties (E/S)** : Pilote les périphériques via les drivers ; gère les interruptions, les tampons et le système de fichiers.

4.2) Trois types de noyau

Type de noyau	Caractéristiques	Exemple de SE
Noyau monolithique (Monolithic kernel)	Tous les services SE s'exécutent dans un espace privilégié. Performances élevées m	Linux, Unix (BSD)
Micro-noyau (Microkernel)	Seuls les services essentiels (IPC, gestion mémoire) sont dans le noyau. Les autres services	MINIX, QNX, Mach
Noyau hybride (Hybrid kernel)	Compromis : noyau relativement compact mais certains services (pilotes, système de fichiers) restent	Windows NT, Solaris, OS X

Exercice 5 : Mémoire

5.1) Cinq objectifs de la gestion de la mémoire

- 1. Relocation (relogement)** : Permettre à un processus d'être chargé n'importe où en mémoire physique, grâce à la translation d'adresses.
- 2. Protection** : Empêcher un processus d'accéder à l'espace mémoire d'un autre processus ou du noyau (violations détectées par le matériel).
- 3. Partage** : Autoriser plusieurs processus à partager des zones mémoire communes (bibliothèques partagées, mémoire partagée IPC).
- 4. Organisation logique** : Fournir une vision organisée et structurée de la mémoire au programmeur (segments de code, de données, pile), indépendamment de l'organisation physique.
- 5. Organisation physique** : Gérer efficacement le transfert d'informations entre la mémoire principale (RAM) et la mémoire secondaire (disque) pour optimiser l'utilisation de l'espace physique disponible.

5.2) Rôle de la mémoire virtuelle

La mémoire virtuelle est un mécanisme qui permet à un processus d'utiliser un espace d'adressage plus grand que la mémoire physique disponible. Ses rôles essentiels sont :

- **Abstraction** : chaque processus dispose de son propre espace d'adressage virtuel complet (ex. 0 à $2^{64}-1$ en 64 bits), indépendant de la RAM réelle.
- **Extension de la mémoire** : les pages non utilisées sont stockées sur disque (swap) et ramenées en RAM à la demande (pagination à la demande / demand paging).
- **Isolation et protection** : deux processus ne peuvent pas accéder directement à la mémoire physique de l'autre ; la MMU (Memory Management Unit) traduit les adresses virtuelles en adresses physiques.
- **Partage facilité** : des pages virtuelles distinctes de deux processus peuvent pointer sur la même page physique (ex. bibliothèques partagées).

